



Data Engineering

Roadmap by Gonalo Verssimo

“Dominar os dados é dominar o futuro”

Este guia foi criado como parte do meu percurso de transição profissional de Data Analyst para Data Engineer. Com alguma experiência em análise de dados, senti a necessidade de aprofundar competências em áreas como integração, modelação, automação e infraestrutura de dados. Desenvolvi este curso como projeto pessoal de autoestudo, estruturado para ser prático, direto e aplicável. Inclui explicações, exemplos e exercícios resolvidos, pensados para facilitar a aprendizagem progressiva.

Índice

1.	Fundamentos.....	5
	Conceitos Essenciais - Linguagem de Programação: Python	5
	Bibliotecas Fundamentais para Engenharia de Dados.....	5
2.	SQL e Modelagem de Dados	7
	Conceitos Essenciais de SQL	7
	Conceitos Essenciais de Modelação de Dados.....	7
3.	Big Data e Computação na Nuvem	9
	Os 5 V's do Big Data	9
	Plataforma Cloud Azure para Engenharia de Dados	9
	Ecossistema Hadoop.....	10
	Comparação: Hadoop vs Spark.....	10
4.	Processamento de Dados.....	11
	Fases típicas de processamento (ETL)	11
	Apache Spark	11
5.	Big Data e Engenharia de Dados Avançada.....	14
	Otimização no Apache Spark	14
	Boas Práticas	14
	Processamento Batch	15
	Processamento Streaming.....	15
6.	Datawarehouse na Nuvem.....	16
	Arquitetura Moderna na Nuvem - Componentes típicos	16
	Plataformas Principais	16
	Comparação	17
	Arquiteturas: Data Lake, Warehouse e Lakehouse	17
7.	Data Streaming.....	18
	Diferença entre Batch e Streaming.....	18
	Apache Kafka.....	18
	Casos Reais de Uso	19
8.	Transformação e Orquestração de Dados	21

Transformação de Dados	21
Ferramentas comuns de transformação	21
Orquestração de Dados	21
Apache Airflow	21
9. Dashboards e Visualização	22
Ferramentas Principais	22
10. DataOps e DevOps	23
DevOps	23
DataOps.....	23
Docker	23
CI/CD	23
DataOps: Práticas recomendadas	24
11. Conceitos Avançados	25
Data Lakehouse	25
Data Fabric	25
Data Mesh.....	25
Vantagens e Desvantagens	26
12. Governança e Observabilidade	27
Governança de Dados	27
Observabilidade	27
Garantia da Qualidade dos Dados	27

1. Fundamentos

Conceitos Essenciais - Linguagem de Programação: Python

Conceito	Descrição
Variáveis	Guardam dados na memória. Ex: nome = "Ana"
Tipos de Dados	int, float, str, list, dict, bool, etc.
Operadores	Aritméticos (+, -, *, /), lógicos (and, or, not), comparação (==, !=, <, >)
Condicionais	Permitem tomar decisões. Ex: if idade > 18:
Ciclos	Iteram sobre listas, dicionários. Ex: for cliente in clientes:
Funções	Reutilização de código. Ex: def calcular_total(lista):
List Comprehensions	Forma concisa de construir listas. Ex: [x for x in lista if x > 0]
Erros e Exceções	Deteção e tratamento de falhas. Ex: try/except
Orientação a Objetos	Uso de classes e métodos. Ex: class Cliente:

Bibliotecas Fundamentais para Engenharia de Dados

Biblioteca	Finalidade	Exemplo
pandas	Manipulação de tabelas (DataFrames)	df = pd.read_csv("dados.csv")
numpy	Operações numéricas, arrays	np.mean([1, 2, 3])

Exemplo de um Script Simples de Transformação

```
1  import pandas as pd
2
3  # Ler CSV
4  df = pd.read_csv("vendas.csv")
5
6  # Calcular total por cliente
7  total_por_cliente = df.groupby("cliente")["valor"].sum().reset_index()
8
9  # Guardar para novo ficheiro
10 total_por_cliente.to_csv("total_clientes.csv", index=False)
```

Exercício 1: Criar uma função em Python que receba uma lista de vendas e retorne o total.

```
1 def calcular_total(vendas):
2     return sum(vendas)
3 vendas = [100, 250, 85, 40]
4 print("Total das vendas:", calcular_total(vendas))
```

Exercício 2: Criar uma lista de dicionários com dados de clientes e usar list comprehension para filtrar apenas os maiores de 30 anos.

```
6 clientes = [
7     {"nome": "Carlos", "idade": 28},
8     {"nome": "Joana", "idade": 35},
9     {"nome": "Miguel", "idade": 40}
10 ]
11 maiores_de_30 = [c for c in clientes if c["idade"] > 30]
12 print(maiores_de_30)
```

Exercício 3: Ler um ficheiro CSV com pandas, filtrar linhas e guardar novo CSV.

```
14 import pandas as pd
15 df = pd.read_csv("produtos.csv")
16 produtos_caros = df[df["preco"] > 100]
17 produtos_caros.to_csv("produtos_filtrados.csv", index=False)
```

Exercício 4: Simular uma leitura de dados, aplicar transformação e gravar num Excel.

```
19 import pandas as pd
20 dados = {
21     "cliente": ["Ana", "Bruno", "Carlos"],
22     "vendas": [250, 300, 150]
23 }
24 df = pd.DataFrame(dados)
25 df["bónus"] = df["vendas"] * 0.05
26 df.to_excel("relatorio_clientes.xlsx", index=False)
```

Exercício 5: Automatizar o processo com uma função processar_csv.

```
28 import pandas as pd
29 def processar_csv(ficheiro_entrada, ficheiro_saida):
30     df = pd.read_csv(ficheiro_entrada)
31     df_filtrado = df[df["valor"] > 100]
32     df_filtrado.to_csv(ficheiro_saida, index=False)
33 processar_csv("vendas.csv", "vendas_filtradas.csv")
```

2. SQL e Modelagem de Dados

Conceitos Essenciais de SQL

Conceito	Descrição
SELECT	Consulta dados de uma tabela. Exemplo: <code>SELECT * FROM clientes;</code>
WHERE	Filtra linhas por condição. Exemplo: <code>WHERE idade > 30</code>
JOIN	Combina dados de duas ou mais tabelas. Tipos: INNER, LEFT, RIGHT, FULL
GROUP BY	Agrupa linhas para agregação. Exemplo: <code>GROUP BY cliente</code>
ORDER BY	Ordena resultados. Exemplo: <code>ORDER BY data_venda DESC</code>
INSERT	Insere novos dados. Exemplo: <code>INSERT INTO clientes VALUES (...)</code>
UPDATE	Atualiza dados existentes. Exemplo: <code>UPDATE clientes SET idade = 31 WHERE id = 1</code>
DELETE	Remove dados. Exemplo: <code>DELETE FROM clientes WHERE id = 10</code>
Funções Agregadas	COUNT(), SUM(), AVG(), MIN(), MAX()
Subqueries	Consultas dentro de consultas para filtrar ou calcular dados

Conceitos Essenciais de Modelação de Dados

Conceito	Descrição
Entidade	Representa um objeto real (ex: Cliente, Produto)
Atributo	Característica da entidade (ex: nome, idade)
Chave Primária	Identificador único da entidade (ex: id_cliente)
Chave Estrangeira	Liga duas entidades, referenciando a chave primária da outra tabela
Normalização	Processo de organizar os dados para evitar redundância e inconsistências
1ª Forma Normal (1FN)	Sem dados repetidos e com valores indivisíveis por coluna.

2ª Forma Normal (2FN)	Cada dado depende da totalidade da chave primária.
3ª Forma Normal (3FN)	Cada dado depende só da chave primária, não de outros dados.

Exercício 1: Selecionar todos os clientes com idade superior a 25 anos.

```
9  SELECT * FROM clientes
10 WHERE idade > 25;
```

Exercício 2: Selecionar o nome do cliente e o valor das vendas realizadas, juntando as tabelas clientes e vendas.

```
13 SELECT a.nome, b.valor
14 FROM clientes a
15 INNER JOIN vendas b ON a.id = b.cliente_id;
```

Exercício 3: Calcular o total de vendas por cliente.

```
19 SELECT cliente_id, SUM(valor) AS total_vendas
20 FROM vendas
21 GROUP BY cliente_id;
```

Exercício 4: Atualizar o email do cliente com id = 3.

```
24 UPDATE clientes
25 SET email = 'novoemail@exemplo.com'
26 WHERE id = 3;
```


3. Big Data e Computação na Nuvem

Big Data refere-se a conjuntos de dados muito grandes e complexos que não podem ser processados por métodos tradicionais.

Os 5 V's do Big Data

V	Significado
Volume	Quantidade massiva de dados gerados
Velocidade	Rapidez com que os dados são gerados e processados
Variedade	Diferentes tipos de dados: estruturados, não estruturados, semi-estruturados
Veracidade	Qualidade e confiança dos dados
Valor	Extração de informação útil a partir dos dados

Exemplo: Um sistema de sensores de uma cidade inteligente (smart city) pode gerar gigabytes de dados por hora. O armazenamento, o processamento e a análise eficiente desse volume só é possível com uma arquitectura Big Data.

Plataforma Cloud Azure para Engenharia de Dados

Serviço Azure	Função
Azure Data Lake	Armazenamento escalável de ficheiros em larga escala
Azure Synapse	Plataforma analítica unificada (Data Warehouse + Big Data)
Azure Data Factory	Orquestração de pipelines ETL/ELT
Azure Databricks	Ambiente colaborativo com Apache Spark para análise de dados
Azure Event Hubs	Processamento de eventos em tempo real
Azure Blob Storage	Armazenamento objectual para ficheiros

Ecosistema Hadoop

Hadoop é um ecossistema open-source criado para armazenamento e processamento distribuído de grandes volumes de dados, com base em clusters de computadores.

Componentes Principais

Componente	Função
HDFS	Sistema de ficheiros distribuído
MapReduce	Modelo de processamento distribuído em paralelo
YARN	Gerenciamento de recursos e agendamento de tarefas
Hive / Pig	Linguagens para consulta e transformação de dados
HBase	Base de dados NoSQL para grandes volumes de dados

Comparação: Hadoop vs Spark

Recurso	Hadoop (MapReduce)	Spark
Velocidade	Mais lento	Muito mais rápido (em memória)
Linguagem	Java (principalmente)	Suporta Scala, Python, SQL
Facilidade	Mais complexo	API mais simples e amigável
Popularidade	Em queda	Muito utilizado em cloud e produção

4. Processamento de Dados

O processamento de dados é o conjunto de operações realizadas para transformar dados brutos em informação útil, estruturada e disponível para análise.

Fases típicas de processamento (ETL)

Etapa	Descrição
E - Extract	Recolha dos dados de diversas fontes (bases de dados, APIs, ficheiros)
T - Transform	Limpeza, agregação, junção e enriquecimento dos dados
L - Load	Gravação dos dados no destino final (data warehouse, lake, etc.)

Apache Spark

O Apache Spark é um motor de processamento distribuído usado para trabalhar com grandes volumes de dados em paralelo, muito mais rápido que Hadoop MapReduce, especialmente com DataFrames e processamento em memória.

Exemplo de um PySpark DataFrame

```
50 from pyspark.sql import SparkSession
51 spark = SparkSession.builder.appName("Exemplo").getOrCreate()
52 dados = [("João", 30), ("Maria", 25), ("Pedro", 28)]
53 colunas = ["nome", "idade"]
54 df = spark.createDataFrame(dados, colunas)
55 df.show()
```

Resultado:

nome	idade
João	30
Maria	25
Pedro	28

Transformações comuns com Spark

```
60 # Filtrar por idade
61 df.filter(df.idade > 26).show()
62
63 # Seleccionar columnas
64 df.select("nome").show()
65
66 # Agregações
67 df.groupBy("idade").count().show()
```

Exemplo com NumPy

```
71 import numpy as np
72 a = np.array([1, 2, 3])
73 b = np.array([4, 5, 6])
74 soma = a + b # Resultado: array([5, 7, 9])
```

Exemplo com Pandas

```
78 import pandas as pd
79 df = pd.DataFrame({
80     "nome": ["João", "Maria", "Pedro"],
81     "idade": [30, 25, 28]
82 })
83 # Filtrar
84 df[df["idade"] > 26]
```

Exemplo ETL com Python (sem Spark)

```
88 import pandas as pd
89
90 # ETAPA 1: Extração
91 df = pd.read_csv("clientes.csv")
92
93 # ETAPA 2: Transformação
94 df["nome"] = df["nome"].str.upper()
95 df = df[df["idade"] > 18]
96
97 # ETAPA 3: Carga
98 df.to_csv("clientes_filtrados.csv", index=False)
```

Exercício 1: Criar um DataFrame com as colunas produto e preço, contendo três produtos.

```
101 dados = [("Camisola", 19.90), ("Calças", 39.90), ("Sapatos", 59.90)]
102 colunas = ["produto", "preço"]
103 df = spark.createDataFrame(dados, colunas)
104 df.show()
```

Exercício 2: A partir do DataFrame de produtos, calcular o preço médio.

```
108 df.groupBy().avg("preço").show()
```

Exercício 3: Ler um ficheiro CSV, transformar todos os nomes para maiúsculas e gravar o resultado noutro ficheiro.

```
113 df = pd.read_csv("clientes.csv")
114 df["nome"] = df["nome"].str.upper()
115 df.to_csv("clientes_editados.csv", index=False)
```

5. Big Data e Engenharia de Dados Avançada

Otimização no Apache Spark

Catalyst Optimizer: O Catalyst é o motor de otimização do Spark que transforma o código em planos de execução eficientes.

Tungsten: Projeto de execução que melhora o uso de CPU e memória com operações em baixo nível.

Boas Práticas

Estratégia	Explicação
cache() ou persist()	Evita recalcular DataFrames que são usados várias vezes
repartition() / coalesce()	Controla o número de partições (evita skew e melhora paralelismo)
broadcast() joins	Distribui pequenas tabelas para todos os nós, evitando shuffles

Exemplo Prático da Otimização de Join

```
119 from pyspark.sql.functions import broadcast
120 df1 = spark.read.csv("clientes.csv", header=True)
121 df2 = spark.read.csv("compras.csv", header=True)
122 # Otimização: broadcast da tabela pequena
123 df_join = df2.join(broadcast(df1), "cliente_id")
124 df_join.show()
```

Jobs Spark

Um Job no Spark é o resultado da execução de uma ação (por ex: `.show()`, `.write()`). Cada job é dividido em stages, e cada stage em tasks.

Ferramentas de Workflow

Ferramenta	Finalidade
Apache Airflow	Orquestrar pipelines ETL com agendamento
Azure Data Factory	Orquestração visual na Azure
Luigi	Orquestração mais simples com Python

Exemplo de um pipeline com Spark e agendamento (pseudo-código Airflow)

```
128 from airflow import DAG
129 from airflow.operators.bash import BashOperator
130 dag = DAG("pipeline_dados", schedule_interval="@daily")
131 job = BashOperator(
132     task_id="run_spark_job",
133     bash_command="spark-submit script.py",
134     dag=dag
135 )
```

Processamento Batch

Processa grandes volumes de dados em blocos, geralmente com agendamento, por exemplo um relatório diário de vendas com dados de ontem.

Processamento Streaming

Processa dados em tempo real ou quase em tempo real, por exemplo leitura contínua de dados de sensores, logs de aplicações, etc.

Exemplo de Spark Structured Streaming

```
139 df_stream = spark.readStream.format("csv") \
140     .option("path", "/dados/novos") \
141     .option("header", True).load()
142 df_filtrado = df_stream.filter("valor > 100")
143 query = df_filtrado.writeStream.format("console").start()
144 query.awaitTermination()
```

Exercício 1: Quer-se juntar uma tabela grande com uma pequena (referência). Como se otimiza a operação?

R: Usar `broadcast()` para a tabela pequena:

```
148 df_final = df_grande.join(broadcast(df_pequena), "chave")
```

Exercício 2: Está-se a recolher dados de cliques de um website em tempo real. Qual modelo se usaria?

R: Streaming, usando Spark Structured Streaming ou ferramentas como Kafka + Spark.

6. Datawarehouse na Nuvem

Um **Data Warehouse** é um sistema otimizado para leitura e análise de grandes volumes de dados estruturados, proveniente de várias fontes (bases de dados operacionais, ficheiros, APIs, etc.). É ideal para: Relatórios e dashboards, BI e análises e suporte à decisão.

Arquitetura Moderna na Nuvem - Componentes típicos

Componente	Função
Ingestão	Leitura de dados brutos de várias fontes (batch/streaming)
Armazenamento	Data lake (ex: S3) ou cloud storage
Transformação	ELT com Spark, dbt, SQL, etc.
Data Warehouse	Redshift, Snowflake, BigQuery, Databricks
Visualização	Power BI, Tableau, Looker, etc.

Plataformas Principais

Amazon Redshift

- Data warehouse da AWS;
- Baseado em PostgreSQL;
- Suporta ELT com integração com S3, Glue, etc.

Snowflake

- Totalmente cloud (não há gestão de infraestrutura);
- Armazenamento e computação são separados;
- Suporta dados semi-estruturados (JSON, XML).

Databricks

- Plataforma unificada para engenharia, ciência e análise de dados;
- Suporta notebooks com Spark, SQL, ML e dashboards;
- Ideal para arquiteturas **Lakehouse**.

Comparação

Plataforma	Vantagem	Ideal para
Redshift	Integração com AWS, velocidade	BI clássico
Snowflake	Simples, escalável, multi-cloud	Empresas com dados híbridos
Databricks	Versátil, inclui ML, Spark nativo	Projectos avançados, Lakehouse

Arquiteturas: Data Lake, Warehouse e Lakehouse

Modelo	Explicação
Data Lake	Armazena dados brutos em cloud (S3, ADLS); não estruturado
Data Warehouse	Dados estruturados e preparados para análise; schema definido
Lakehouse	Combina os dois — estrutura de warehouse com flexibilidade de lake (Databricks)

Exercício 2: Tem-se dados de sensores em tempo real e precisa-se de fazer uma análise histórica e streaming. Que plataforma se escolheria?

R: Databricks, por suportar batch e streaming, além de análise com Spark.

Exercício 3: Desenhe uma arquitetura simples com ingestão de dados para um DWH na nuvem.

```
54 [Fontes de Dados: APIs / Ficheiros / Bases de Dados On-Prem ou na Nuvem]
55     ↓
56 [Ingestão: Azure Data Factory (ADF) / AWS Glue / Fivetran / Airbyte]
57     ↓
58 [Data Lake: ADLS (Azure) / S3 (AWS) – armazenamento raw]
59     ↓
60 [Transformação: Spark (Databricks) / dbt / Glue Jobs]
61     ↓
62 [DWH: Snowflake / Redshift / Databricks SQL / BigQuery]
63     ↓
64 [Visualização: Power BI / Tableau / Looker / Excel]
```

7. Data Streaming

Data Streaming é o processamento contínuo de dados que chegam em tempo real ou quase em tempo real, como cliques num website, transações financeiras, logs, sensores IoT, etc.

Diferença entre Batch e Streaming

Processamento Batch	Processamento Streaming
Executa sobre blocos de dados fixos	Executa em tempo real, conforme os dados chegam
Maior latência, ideal para BI tradicional	Baixa latência, ideal para eventos em tempo real
Exemplos: relatórios diários	Exemplos: alerta de fraude, logs em tempo real

Apache Kafka

O Apache Kafka é uma plataforma distribuída de streaming que permite:

- Publicar e subscrever fluxos de dados;
- Armazenar mensagens de forma distribuída;
- Lidar com milhões de eventos por segundo.

Componentes Principais

Componente	Descrição
Producer	Envia mensagens para um tópico
Broker	Nó do cluster Kafka que recebe e distribui mensagens
Topic	Canal onde as mensagens são agrupadas (por tema)
Consumer	Lê mensagens dos tópicos
Zookeeper	Gere o estado do cluster (pode ser substituído por KRaft)

Exemplo Kafka CLI

```
38 # Criar um tópico
39 kafka-topics.sh --create --topic sensores --bootstrap-server localhost:9092
40
41 # Enviar mensagem (producer)
42 kafka-console-producer.sh --topic sensores --bootstrap-server localhost:9092
43
44 # Ler mensagens (consumer)
45 kafka-console-consumer.sh --topic sensores --from-beginning --bootstrap-server localhost:9092
46
```

Integração com Spark Streaming

O **Spark Structured Streaming** pode consumir dados diretamente de Kafka.

Exemplo com PySpark

```
154 df = spark.readStream \
155     .format("kafka") \
156     .option("kafka.bootstrap.servers", "localhost:9092") \
157     .option("subscribe", "sensores") \
158     .load()
159 df_valores = df.selectExpr("CAST(value AS STRING)")
```

Casos Reais de Uso

- Detecção de fraudes em tempo real;
- Monitorização de aplicações;
- Processamento de logs;
- Análise de cliques em websites;
- Dados de sensores em tempo real (IoT).

Exercício 1: Qual a vantagem de usar Apache Kafka numa arquitetura de dados moderna?

R: Kafka permite tratar e transportar grandes volumes de dados em tempo real, com fiabilidade e tolerância a falhas, ideal para pipelines de streaming escaláveis.

Exercício 2: O que é um tópico no Kafka? Como se compara a uma tabela de base de dados?

R: Um tópico é um canal de mensagens categorizadas por assunto. Cada mensagem é armazenada ordenadamente. Ao contrário das tabelas, não tem esquema fixo nem transações complexas, é orientado a eventos.

Exercício 3: Simular o envio de dados de temperatura e ler esses dados em tempo real.

```
68 # Terminal 1 (Producer)
69 kafka-console-producer.sh --topic temperatura --bootstrap-server localhost:9092
70 > 23.4
71 > 24.1
72 > 22.7
73
74 # Terminal 2 (Consumer)
75 kafka-console-consumer.sh --topic temperatura --from-beginning --bootstrap-server localhost:9092
76
```

8. Transformação e Orquestração de Dados

Transformação de Dados

A transformação é o processo de modificar dados brutos para um formato útil e estruturado, adequado para análise, reporting ou outras operações. Pode incluir limpeza, agregação, filtragem, enriquecimento, etc.

Ferramentas comuns de transformação

Ferramenta	Tipo	Descrição
Python (Pandas, PySpark)	Linguagem / Framework	Manipulação e transformação programática
dbt (Data Build Tool)	Ferramenta SQL-based	Transformações SQL versionadas e organizadas
Apache Spark	Framework distribuído	Processamento em larga escala
Azure Data Factory	Plataforma ETL/ELT	Interface visual para pipelines

Orquestração de Dados

Orquestração é o processo de automação, gestão e monitorização dos pipelines de dados. Permite ligar várias tarefas (ETL, transformações, cargas, testes) numa sequência lógica e controlada.

Apache Airflow

- Plataforma open source para orquestração de workflows;
- Baseada em DAGs (Directed Acyclic Graphs) que definem as tarefas e dependências;
- Permite agendamento, monitorização, alertas e escalabilidade.

Exemplo simples de DAG no Airflow

```
165 from airflow import DAG
166 from airflow.operators.bash import BashOperator
167 from datetime import datetime
168 dag = DAG('exemplo_orquestracao', start_date=datetime(2025,7,1), schedule_interval='@daily')
169 tarefa_1 = BashOperator(task_id='extrair_dados', bash_command='python extrair.py', dag=dag)
170 tarefa_2 = BashOperator(task_id='transformar_dados', bash_command='python transformar.py', dag=dag)
171 tarefa_3 = BashOperator(task_id='carregar_dados', bash_command='python carregar.py', dag=dag)
172 tarefa_1 >> tarefa_2 >> tarefa_3
```

9. Dashboards e Visualização

Princípios básicos:

- Simplicidade: evitar excesso de informação;
- Claridade: escolher gráficos adequados ao tipo de dados;
- Consistência: cores, legendas e formatos uniformes;
- Contexto: explicar sempre o que o gráfico mostra.

Ferramentas Principais

Ferramenta	Características	Utilização comum
Power BI	Integrado com Microsoft, fácil de usar, gratuito em versão básica	Dashboards empresariais, relatórios
Tableau	Visualizações sofisticadas, forte comunidade	Análise avançada, storytelling

10. DataOps e DevOps

DevOps

É uma cultura e conjunto de práticas que une desenvolvimento (Dev) e operações (Ops) para acelerar a entrega de software com qualidade. Inclui:

- Automação de build, testes e deploy;
- Monitorização contínua;
- Colaboração entre equipas.

DataOps

É a aplicação dos princípios DevOps ao ciclo de vida dos dados, focando em:

- Automação de pipelines de dados;
- Monitorização e qualidade dos dados;
- Gestão colaborativa e ágil de dados.

Docker

É uma plataforma para criar, distribuir e executar aplicações em containers, ambientes isolados e leves que contêm tudo o que a aplicação precisa para funcionar.

Vantagens de Docker em Data Engineering:

- Portabilidade: mesma imagem roda em qualquer ambiente;
- Isolamento: evita conflitos entre dependências;
- Escalabilidade: fácil replicar e gerir containers.

Exemplo básico de Dockerfile para um pipeline Python

```
78 FROM python:3.9-slim
79 WORKDIR /app
80 COPY requirements.txt .
81 RUN pip install -r requirements.txt
82 COPY . .
83 CMD ["python", "pipeline.py"]
```

CI/CD

- **CI (Continuous Integration):** integração frequente de código com testes automáticos.
- **CD (Continuous Delivery/Deployment):** entrega automática para produção ou ambientes de testes.

Ferramentas comuns

Ferramenta	Função
Jenkins	Automação de pipelines de build/deploy
GitHub Actions	CI/CD integrado com GitHub
Azure DevOps	Plataforma completa de DevOps

DataOps: Práticas recomendadas

- Versionar código e scripts;
- Monitorizar qualidade e integridade dos dados;
- Automatizar testes de dados (ex: validar formatos, valores, duplicados);
- Implementar alertas e logging detalhado.

Exercício 1: Criar um Dockerfile para um script Python que usa a biblioteca pandas.

```
87 FROM python:3.9-slim
88 WORKDIR /app
89 COPY requirements.txt .
90 RUN pip install -r requirements.txt
91 COPY . .
92 CMD ["python", "script.py"]
93 -- E o ficheiro requirements.txt deve conter: pandas
```

Exercício 2: Explicar o que acontece num pipeline CI básico após um push para o repositório.

R: O código é automaticamente puxado, compilado, e os testes são executados para garantir que tudo funciona antes de ser aceite no ramo principal.

11. Conceitos Avançados

Data Lakehouse

Uma arquitetura que combina as vantagens dos Data Lakes (armazenamento de dados brutos) e dos Data Warehouses (estruturação, performance para BI). Permite processamento unificado, quer para dados estruturados, quer para não estruturados.

Características:

- Armazenamento económico e escalável (ex: AWS S3, Azure Data Lake);
- Suporte a esquemas flexíveis e evolutivos e a processamento transacional e análises rápidas;
- Ferramentas comuns: Delta Lake, Apache Iceberg, Apache Hudi.

Data Fabric

É uma arquitetura que oferece uma camada unificada e inteligente para gerir dados dispersos em diferentes ambientes (on-premises, cloud, multi-cloud), facilitando acesso, integração e governança.

Características:

- Integra dados de múltiplas fontes;
- Usa machine learning para automatizar gestão de dados;
- Fornece visibilidade e segurança centralizada;
- Foca na automação e gestão dinâmica.

Data Mesh

É uma abordagem descentralizada para gestão de dados, onde os domínios de negócio são responsáveis pelos seus próprios dados como produtos, promovendo autonomia e escalabilidade.

Características:

- Organização baseada em domínios (ex: vendas, marketing);
- Cada domínio gere os seus dados (autonomia);
- Interfaces padronizadas para consumo dos dados;
- Promove cultura orientada a dados.

Vantagens e Desvantagens

Arquitetura	Vantagens	Desvantagens
Data Lakehouse	Flexibilidade, unificação de dados, performance	Pode ser complexa para implementar corretamente
Data Fabric	Visibilidade central, automação	Complexidade tecnológica e custos iniciais
Data Mesh	Escalabilidade, autonomia, foco no negócio	Requer cultura organizacional forte e maturidade

12. Governança e Observabilidade

Governança de Dados

Governança de dados é o conjunto de processos, políticas e padrões que asseguram que os dados são geridos de forma segura, correta e eficiente, respeitando a privacidade e as normas legais.

Componentes principais:

- **Segurança:** controlar acesso e proteger contra ameaças;
- **Privacidade:** garantir conformidade com GDPR e outras leis;
- **Qualidade dos dados:** assegurar que os dados são precisos, completos e atualizados;
- **Responsabilidades:** definir papéis claros (data owners, stewards).

Observabilidade

É a capacidade de monitorizar e entender o estado dos pipelines e sistemas de dados em produção, através de métricas, logs e alertas.

Práticas comuns:

- Monitorização de processos ETL/ELT;
- Alertas em falhas ou desvios de qualidade;
- Logging detalhado para debugging;
- Dashboards de saúde do sistema.

Garantia da Qualidade dos Dados

- Implementar validações automáticas (ex: formatos, valores, duplicados);
- Uso de ferramentas como Great Expectations, Deequ;
- Estabelecer SLAs para a entrega de dados;
- Revisão e auditoria periódica.